

СОДЕРЖАНИЕ

1. ИЗМЕНЕНИЯ, КОТОРЫЕ БЫЛИ ПРОВЕДЕНЫ В ИСХОДНОМ КОДЕ ПРОЕКТА.....	2
2. СКРИНШОТЫ РЕЗУЛЬТАТОВ, ПОЛУЧЕННОГО В П.5-8 ЗАДАНИЙ .	6
3. Ответы на вопросы.....	8
ВЫВОД.....	10

1. ИЗМЕНЕНИЯ, КОТОРЫЕ БЫЛИ ПРОВЕДЕНЫ В ИСХОДНОМ КОДЕ ПРОЕКТА

1. Найти отсутствующую зависимость и указать ее в соответствующем блоке в build.gradle, чтобы проект снова начал собираться

```
23 dependencies {
24     annotationProcessor 'org.projectlombok:lombok:1.18.18'
25     compileOnly 'org.projectlombok:lombok:1.18.18'
26
27     implementation("io.micronaut:micronaut-runtime")
28     implementation("io.micronaut:micronaut-validation")
29     implementation("io.micronaut:micronaut-http-client")
30     implementation("javax.annotation:javax.annotation-api")
31     implementation("org.apache.logging.log4j:log4j-core:2.12.1")
32     implementation("com.opencsv:opencsv:5.9")
33     runtimeOnly("org.apache.logging.log4j:log4j-api:2.12.1")
34     runtimeOnly("org.apache.logging.log4j:log4j-slf4j-impl:2.12.1")
35 }
```

Рисунок 1 – Изменения в файле build.gradle

2. В некоторых классах поправить имя пакета

```
3 import com.opencsv.bean.CsvToBeanBuilder;
4 import io.micronaut.http.HttpResponse;
5 import io.micronaut.http.annotation.Controller;
6 import io.micronaut.http.annotation.Get;
7 import ru.mirea.trppsecond6.entity.Message;
```

Рисунок 2 – Изменения в файле BookContorller.java

```
1 package ru.mirea.trppsecond6.controllers;
2
3 import io.micronaut.http.HttpResponse;
4 import io.micronaut.http.annotation.Controller;
5 import io.micronaut.http.annotation.Get;
6 import ru.mirea.trppsecond6.entity.HealthResponse;
```

Рисунок 3 – Изменения в файле HealthController.java

Запустим проект с исправленными именами пакетов. Видим, что все собралось без проблем

```
> Task :compileTestJava
Note: Creating bean classes for 1 type elements
warning: No processor claimed any of these annotations: /io.micronautTest,/javax.inject.Inject,/org.junit.jupiter.api.Test
1 warning

BUILD SUCCESSFUL in 16s
16 actionable tasks: 10 executed, 6 up-to-date
```

Рисунок 4 – Вывод консоли

3. Собрать документацию проекта, найти в ней запросы состояния и сущности по идентификатору

```
PS C:\Users\Redmi\Desktop\trpp-second-6> ./gradlew javadoc
Starting a Gradle Daemon, 2 busy and 1 incompatible and 1 stopped Daemons
use --status for details

> Task :compileJava
Note: Creating bean classes for 2 type elements
warning: No processor claimed any of these annotations: /io.micronaut.http.er,/io.micronaut.http.annotation.Get,/com.fasterxml.jackson.annotation.Jsonsv.bean.CsvBindByName
1 warning

BUILD SUCCESSFUL in 7s
3 actionable tasks: 3 executed
```

Рисунок 5 – Javadoc

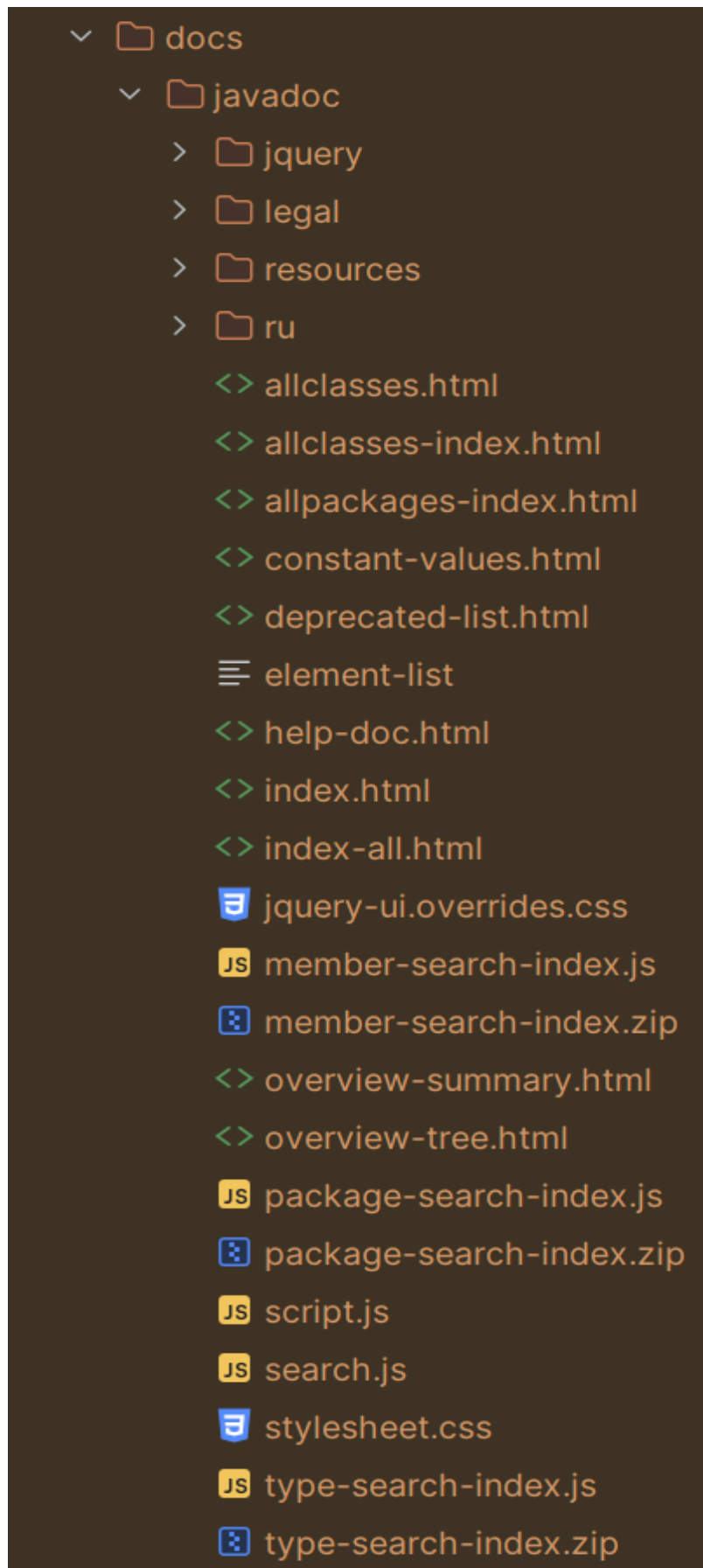


Рисунок 6 – Результат выполнения скрипта

2. СКРИНШОТЫ РЕЗУЛЬТАТОВ, ПОЛУЧЕННОГО В П.5-8 ЗАДАНИЙ

5. Запросить состояние запущенного сервера (GET запрос по адресу <http://localhost:8080>)

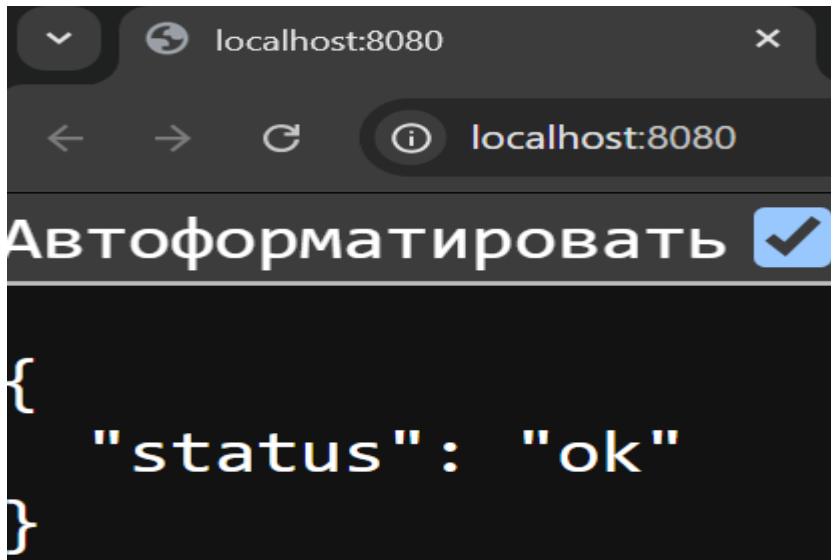


Рисунок 10 – Состояние запущенного сервера

6. Запросить сущность по идентификатору (GET запрос по адресу: <http://localhost:8080/сущность/идентификатор>) (идентификатор – 852)

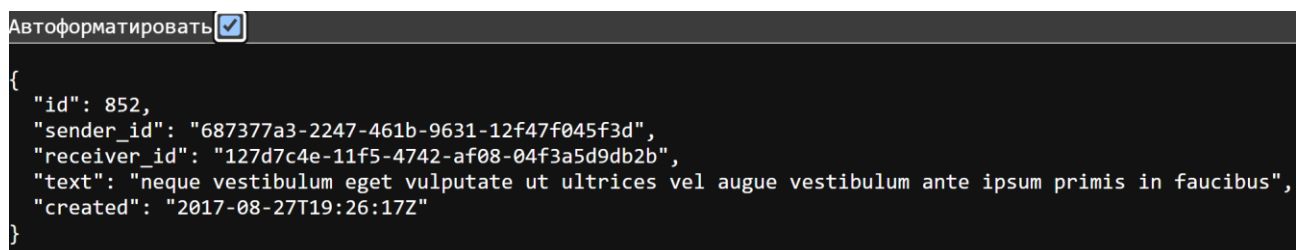


Рисунок 11 – Запрос сущности по идентификатору

7. В задаче shadowJar добавить к jar-файлу вашу фамилию

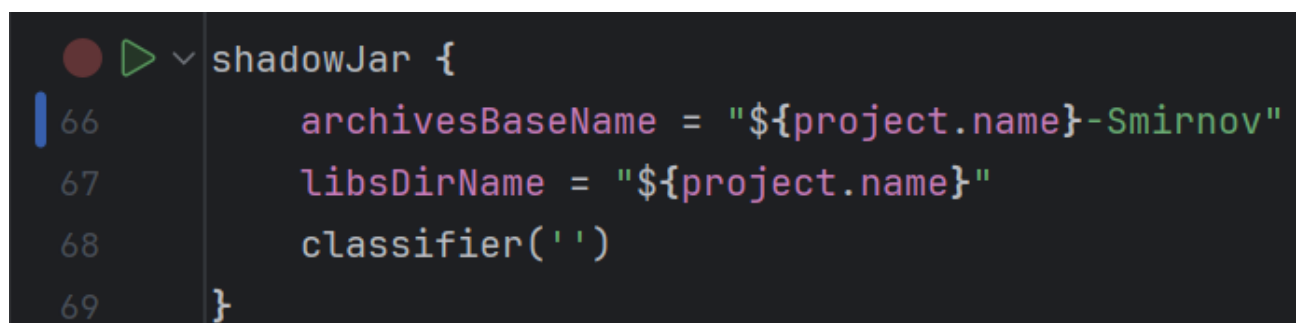


Рисунок 12 – Добавление фамилии к jar-файлу

8. Выполнить задачу checkstyleMain. Посмотреть сгенерированный отчет.
Устранить ошибки оформления кода.

```
PS C:\Users\Redmi\Desktop\trpp-second-6> ./gradlew checkstylemain

> Task :compileJava
Note: Creating bean classes for 2 type elements
warning: A file for type 'ru.mirea.trppsecond6.controllers.$HealthControllerDefinition' already exists on the sourcepath or classpath
warning: A file for type 'ru.mirea.trppsecond6.controllers.$HealthControllerDefinition$$exec1' already exists on the sourcepath or classpath
warning: A file for type 'ru.mirea.trppsecond6.controllers.$HealthControllerDefinitionClass' already exists on the sourcepath or classpath
warning: No processor claimed any of these annotations: /io.micronaut.http.annotation.Controller,/io.micronaut.http.annotation.Get
4 warnings

BUILD SUCCESSFUL in 1s
```

Рисунок 13 – Запуск checkstyleMain

3. Ответы на вопросы

5) Что такое Gradle?

- **Gradle** - это инструмент сборки проектов, который используется в основном для разработки программного обеспечения на языке Java, хотя его также можно использовать и для других языков программирования. Он предоставляет гибкий способ управления проектами и их зависимостями, а также автоматизации сборки, тестирования и развертывания приложений. Gradle использует DSL (Domain Specific Language) на основе Groovy или Kotlin для описания сборочных скриптов, что делает его более удобным и гибким по сравнению с другими инструментами сборки.

6) Что такое Maven?

- **Maven** - это инструмент управления проектами, который обеспечивает сборку, отчетность и управление зависимостями в проектах на Java. Он использует XML-файлы для описания структуры проекта, его зависимостей и сборочных целей. Maven автоматически загружает зависимости из центрального репозитория и обеспечивает стандартизированные структуры проектов, что делает его широко используемым инструментом в сообществе Java.

9) Что делает задача build?

- Задача **build** (сборка) в инструментах сборки, таких как Gradle или Maven, отвечает за компиляцию и сборку исходного кода проекта в исполняемый или дистрибутивный файл. Это включает в себя выполнение различных этапов, таких как компиляция исходного кода, копирование ресурсов, создание исполняемых файлов, упаковку и т. д.

10) Что делает задача compileJava?

- Задача `compileJava` отвечает за компиляцию исходного кода Java в байт-код Java (файлы `.class`). Она часто является частью процесса сборки проекта и выполняется перед другими этапами сборки, такими как создание исполняемых файлов или дистрибутивов.

13) Что такое `javadoc`?

- `Javadoc` - это инструмент, входящий в состав JDK (Java Development Kit), который автоматически генерирует документацию API на основе комментариев в исходном коде Java. Эти комментарии должны следовать определенным соглашениям о стиле и содержать информацию о классах, методах, полях и других элементах API. Сгенерированная документация предоставляет полезную справочную информацию для разработчиков, использующих библиотеку или API.

14) Что такое `checkstyle`?

- `Checkstyle` - это инструмент статического анализа кода для языка Java, который используется для обеспечения согласованности и соответствия кода определенным стандартам оформления. Он проверяет исходный код на соответствие правилам, определенным в конфигурационных файлах, и предоставляет отчет о найденных стилистических или структурных нарушениях. `Checkstyle` помогает обеспечить высокое качество кода, улучшить его читаемость и уменьшить количество потенциальных ошибок.

ВЫВОД

В ходе выполнения работы была проведена диагностика и исправление ошибок в проекте на Java с использованием системы сборки Gradle. В результате данной работы получены практические навыки работы с Gradle, включая настройку зависимостей, управление пакетами, сборку документации, создание UberJar и запуск приложения. Также была освоена проверка кода на соответствие стилю кодирования с помощью Checkstyle.